# UD2.2: Diccionarios, Tuplas y Sets

**Módulo**: Desarrollo de Aplicaciones con Lenguaje de Programación (DALP)

## Índice de contenidos


### 1. Diccionarios

- 1.1. Concepto y sintaxis
- 1.2. Creación y acceso a elementos
- 1.3. Métodos principales
- 1.4. Iteración sobre diccionarios
- 1.5. Diccionarios anidados
- 1.6. Casos de uso prácticos

### 2. Tuplas

- 2.1. Concepto básico e inmutabilidad
- 2.2. Desempaquetado
- 2.3. Cuándo usar tuplas

### 3. Sets

- 3.1. Concepto y eliminación de duplicados
- 3.2. Operaciones de conjuntos
- 3.3. Cuándo usar sets

# 1. Diccionarios

## 1.1. Concepto y sintaxis

Un **diccionario** (dict) es una estructura de datos que almacena pares **clave-valor**. Es la estructura más versátil de Python para organizar datos relacionados.

**Características principales:**

- **Mutable**: Se puede modificar después de su creación
- **Ordenado**: Mantiene el orden de inserción (Python 3.7+)
- **Claves únicas**: Cada clave aparece solo una vez
- **Acceso rápido**: Búsqueda por clave es O(1)

**Sintaxis básica:**

```
# Diccionario vacío
vacio = {}
otro_vacio = dict()

# Diccionario con elementos
persona = {
    "nombre": "Ana",
    "edad": 25,
    "ciudad": "Madrid"
}

# Diferentes tipos de claves
mixto = {
    "texto": "valor",
    42: "número como clave",
    (1, 2): "tupla como clave"
}
```

**¿Por qué usar diccionarios?**

1. **Mapeo de datos**: Asociar información relacionada (nombre → teléfono)
2. **Búsqueda eficiente**: Acceso instantáneo por clave
3. **Estructuración**: Representar objetos complejos
4. **Contadores**: Contar frecuencias de elementos

## 1.2. Creación y acceso a elementos

### Creación:

```
# Literal
estudiante = {
    "nombre": "Carlos",
    "edad": 22,
    "carrera": "Ingeniería",
    "promedio": 8.5
}

# Constructor dict()
persona = dict(nombre="Ana", edad=25, ciudad="Barcelona")

# Desde lista de tuplas
coordenadas = dict([("x", 10), ("y", 20), ("z", 30)])

# Con valores por defecto
desde_keys = dict.fromkeys(["a", "b", "c"], 0)
# {'a': 0, 'b': 0, 'c': 0}
```

### Acceso:

```python
persona = {
    "nombre": "Ana",
    "edad": 25,
    "ciudad": "Madrid"
}

# Acceso directo con [] (error si no existe)
nombre = persona["nombre"]  # "Ana"

# Acceso seguro con .get() (None si no existe)
telefono = persona.get("telefono")  # None
telefono = persona.get("telefono", "No disponible")  # "No disponible"

# Verificar existencia
if "edad" in persona:
    print(f"Edad: {persona['edad']}")

# Modificar
persona["edad"] = 26

# Agregar nuevo elemento
persona["profesion"] = "Ingeniera"

# Eliminar
del persona["profesion"]
```

# 1.3. Métodos principales

## Métodos de acceso

```
persona = {"nombre": "Ana", "edad": 25, "ciudad": "Madrid"}

# keys() - Obtener claves
claves = list(persona.keys())  # ['nombre', 'edad', 'ciudad']

# values() - Obtener valores
valores = list(persona.values())  # ['Ana', 25, 'Madrid']

# items() - Obtener pares (clave, valor)
items = list(persona.items())  # [('nombre', 'Ana'), ('edad', 25), ...]

# get(clave, default) - Acceso seguro
telefono = persona.get("telefono", "No disponible")

# setdefault(clave, default) - Obtiene o crea
edad = persona.setdefault("edad", 0)  # Devuelve 25
email = persona.setdefault("email", "")  # Crea 'email': ''
```

## Métodos de modificación

```
persona = {"nombre": "Ana", "edad": 25}

# update() - Actualizar con otro diccionario
persona.update({"edad": 26, "ciudad": "Madrid"})

# pop(clave) - Eliminar y devolver valor
edad = persona.pop("edad")  # 26

# popitem() - Eliminar último elemento
ultimo = persona.popitem()  # ('ciudad', 'Madrid')

# clear() - Vaciar diccionario
persona.clear()  # {}
```

**Tabla resumen:**

| Método | Descripción | Modifica |
|---|---|---|
| `keys()` | Devuelve claves | No |
| `values()` | Devuelve valores | No |
| `items()` | Devuelve pares | No |
| `get(k, d)` | Valor o default | No |
| `setdefault(k, d)` | Valor o crea | Sí |
| `update(d)` | Actualiza con otro | Sí |
| `pop(k)` | Elimina y devuelve | Sí |
| `clear()` | Vacía diccionario | Sí |

## 1.4. Iteración sobre diccionarios

```python
persona = {"nombre": "Ana", "edad": 25, "ciudad": "Madrid"}

# Iterar sobre claves
for clave in persona:
    print(clave)

# Iterar sobre valores
for valor in persona.values():
    print(valor)

# Iterar sobre items
for clave, valor in persona.items():
    print(f"{clave}: {valor}")

# Con enumerate()
for indice, (clave, valor) in enumerate(persona.items(), start=1):
    print(f"{indice}. {clave}: {valor}")
```

## 1.5. Diccionarios anidados

Los diccionarios pueden contener otros diccionarios, creando estructuras jerárquicas:

```python
# Base de datos de estudiantes
estudiantes = {
    "est001": {
        "nombre": "Ana",
        "edad": 20,
        "notas": [8.5, 9.0, 7.5]
    },
    "est002": {
        "nombre": "Carlos",
        "edad": 22,
        "notas": [7.0, 8.0, 8.5]
    }
}

# Acceder a datos anidados
nombre = estudiantes["est001"]["nombre"]  # "Ana"
primera_nota = estudiantes["est002"]["notas"][0]  # 7.0

# Modificar
estudiantes["est001"]["edad"] = 21

# Agregar nuevo estudiante
estudiantes["est003"] = {
    "nombre": "María",
    "edad": 21,
    "notas": [9.5, 9.0]
}

# Iterar
for id_estudiante, datos in estudiantes.items():
    print(f"{id_estudiante}: {datos['nombre']}")
```

### Ejemplo: Configuración de aplicación

```python
config = {
    "app": {
        "name": "Mi Aplicación",
        "version": "1.0.0",
        "debug": True
    },
    "database": {
        "host": "localhost",
        "port": 5432,
        "user": "admin"
    }
}

# Acceso
db_host = config["database"]["host"]  # "localhost"
app_name = config["app"]["name"]  # "Mi Aplicación"
```

# 1.6. Casos de uso prácticos

## 1. Contador de frecuencias

```python
def contar_palabras(texto):
    palabras = texto.lower().split()
    conteo = {}
    for palabra in palabras:
        conteo[palabra] = conteo.get(palabra, 0) + 1
    return conteo

texto = "python es genial python es poderoso python es fácil"
frecuencias = contar_palabras(texto)
print(frecuencias)
# {'python': 3, 'es': 3, 'genial': 1, 'poderoso': 1, 'fácil': 1}

# Palabra más frecuente
mas_comun = max(frecuencias, key=frecuencias.get)
print(f"Más común: {mas_comun}")
```

## 2. Agrupación de datos

```python
# Agrupar estudiantes por ciudad
estudiantes = [
    {"nombre": "Ana", "ciudad": "Madrid"},
    {"nombre": "Carlos", "ciudad": "Barcelona"},
    {"nombre": "María", "ciudad": "Madrid"}
]

por_ciudad = {}
for estudiante in estudiantes:
    ciudad = estudiante["ciudad"]
    por_ciudad.setdefault(ciudad, []).append(estudiante["nombre"])

print(por_ciudad)
# {'Madrid': ['Ana', 'María'], 'Barcelona': ['Carlos']}
```

## 3. Sistema de inventario

```python
inventario = {
    "PRO001": {"nombre": "Laptop", "precio": 800, "stock": 15},
    "PRO002": {"nombre": "Mouse", "precio": 20, "stock": 50},
    "PRO003": {"nombre": "Teclado", "precio": 45, "stock": 30}
}

# Valor total del inventario
valor_total = 0
for prod in inventario.values():
    valor_total += prod["precio"] * prod["stock"]
print(f"Valor total: ${valor_total}")

# Productos con stock bajo
stock_bajo = {}
for codigo, info in inventario.items():
    if info["stock"] < 20:
        stock_bajo[codigo] = info["nombre"]
print(f"Stock bajo: {stock_bajo}")
```

# 2. Tuplas

## 2.1. Concepto básico e inmutabilidad

Una **tupla** es una colección **inmutable** y **ordenada**. Similar a una lista, pero no se puede modificar.

```
# Sintaxis
tupla_vacia = ()
numeros = (1, 2, 3, 4, 5)
mixta = (42, "texto", 3.14, True)

# Tupla de un elemento (requiere coma)
un_elemento = (5,)  # Correcto
no_tupla = (5)      # Esto es solo un número

# Sin paréntesis (tuple packing)
coordenadas = 10, 20, 30  # Es una tupla

# Acceso por índice
colores = ("rojo", "verde", "azul")
primero = colores[0]    # "rojo"
ultimo = colores[-1]    # "azul"

# Slicing
primeros_dos = colores[:2]  # ('rojo', 'verde')

# NO se puede modificar (inmutable)
# colores[0] = "amarillo"  # Error: las tuplas son inmutables
```

**Características:**

- **Inmutable**: No se puede modificar después de crear
- **Más rápida** que las listas
- Puede usarse como **clave de diccionario**

**Métodos (solo 2):**

```python
numeros = (1, 2, 2, 3, 2, 4)

# count() - Contar ocurrencias
cantidad = numeros.count(2)  # 3

# index() - Primera posición
posicion = numeros.index(3)  # 3
```

## 2.2. Desempaquetado

```python
# Desempaquetado básico
persona = ("Ana", 25, "Madrid")
nombre, edad, ciudad = persona

print(nombre)   # "Ana"
print(edad)     # 25
print(ciudad)   # "Madrid"

# Intercambio de variables
a = 5
b = 10
a, b = b, a  # Intercambio en una línea
print(a, b)  # 10 5

# Desempaquetado con *
numeros = (1, 2, 3, 4, 5, 6, 7, 8)
primero, *resto, ultimo = numeros
print(primero)  # 1
print(resto)    # [2, 3, 4, 5, 6, 7]
print(ultimo)   # 8

# En bucles
puntos = [(1, 2), (3, 4), (5, 6)]
for x, y in puntos:
    print(f"({x}, {y})")

# Funciones que retornan tuplas
def obtener_estadisticas(nums):
    return min(nums), max(nums), sum(nums)/len(nums)

datos = [10, 20, 30]
minimo, maximo, promedio = obtener_estadisticas(datos)
```

## 2.3. Casos de uso de tuplas

### Aplicaciones principales:

1. **Datos inmutables**:

```
CONFIG = ("localhost", 8080, "admin")
DIAS_SEMANA = ("lun", "mar", "mié", "jue", "vie", "sáb", "dom")
```

2. **Retorno múltiple de funciones**:

```
def dividir(a, b):
    return a // b, a % b  # cociente, resto

resultado = dividir(17, 5)  # (3, 2)
```

3. **Claves de diccionario**:

```
coordenadas_ciudades = {
    (40.4168, -3.7038): "Madrid",
    (41.3851, 2.1734): "Barcelona"
}
```

4. **Coordenadas y posiciones**:

```
punto_2d = (10, 20)
rgb = (255, 128, 0)
fecha = (2024, 12, 16)
```

**Tupla vs. Lista:**

| Característica | Tupla | Lista |
|---|---|---|
| Mutabilidad | Inmutable | Mutable |
| Sintaxis | `(1, 2, 3)` | `[1, 2, 3]` |
| Velocidad | Más rápida | Más lenta |
| Casos de uso | Datos fijos | Datos que cambian |
| Como clave dict | ✓ Sí | ✗ No |

# 3. Sets

## 3.1. Concepto y eliminación de duplicados

Un **set** es una colección **no ordenada** de elementos **únicos**.

```
# Sintaxis
set_vacio = set()  # Nota: {} crea un diccionario vacío
numeros = {1, 2, 3, 4, 5}
nombres = {"Ana", "Carlos", "María"}

# Desde lista (elimina duplicados)
lista = [1, 2, 2, 3, 3, 3, 4]
sin_duplicados = set(lista)  # {1, 2, 3, 4}

# Desde string
letras = set("Python")  # {'P', 'y', 't', 'h', 'o', 'n'}

# NO tiene orden ni índices
colores = {"rojo", "verde", "azul"}
# colores[0]  # Error: los sets no se pueden indexar

# Los duplicados se ignoran automáticamente
numeros = {1, 2, 2, 3, 3, 3}
print(numeros)  # {1, 2, 3}
```

**Métodos básicos:**

```python
frutas = {"manzana", "banana"}

# add() - Agregar elemento
frutas.add("naranja")

# remove() - Eliminar (error si no existe)
frutas.remove("banana")

# discard() - Eliminar (sin error si no existe)
frutas.discard("uva")  # No da error

# Verificar pertenencia (muy rápido)
if "manzana" in frutas:
    print("Tenemos manzanas")

# Eliminar duplicados de lista
lista = [1, 2, 2, 3, 3, 4]
sin_dup = list(set(lista))  # [1, 2, 3, 4]
```

## 3.2. Operaciones de conjuntos

```python
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Unión (|) - Todos los elementos
union = a | b  # {1, 2, 3, 4, 5, 6}

# Intersección (&) - Elementos comunes
interseccion = a & b  # {3, 4}

# Diferencia (-) - En A pero no en B
diferencia = a - b  # {1, 2}

# Diferencia simétrica (^) - En A o B, no en ambos
dif_simetrica = a ^ b  # {1, 2, 5, 6}
```

**Ejemplo práctico:**

```python
# Estudiantes en diferentes cursos
python = {"Ana", "Carlos", "María", "Juan"}
javascript = {"María", "Juan", "Pedro", "Luis"}

# Estudiantes en ambos cursos
ambos = python & javascript
print(f"En ambos: {ambos}")  # {'María', 'Juan'}

# Solo en Python
solo_python = python - javascript
print(f"Solo Python: {solo_python}")  # {'Ana', 'Carlos'}

# Total de estudiantes únicos
todos = python | javascript
print(f"Total: {len(todos)}")  # 6
```

## 3.3. Casos de uso de sets

**Aplicaciones principales:**

1. **Eliminación de duplicados**:

```python
def tiene_duplicados(lista):
    return len(lista) != len(set(lista))

print(tiene_duplicados([1, 2, 3]))     # False
print(tiene_duplicados([1, 2, 2, 3]))  # True
```

2. **Búsqueda rápida de pertenencia**:

```python
# Verificación rápida de pertenencia
ids_validos = set(range(1000000))
if 999999 in ids_validos:
    print("ID válido")
```

3. **Operaciones matemáticas de conjuntos**:

```python
# Encontrar elementos únicos en dos listas
lista1 = [1, 2, 3, 4]
lista2 = [3, 4, 5, 6]

comunes = set(lista1) & set(lista2)  # {3, 4}
solo_l1 = set(lista1) - set(lista2)  # {1, 2}
todos = set(lista1) | set(lista2)    # {1, 2, 3, 4, 5, 6}
```

4. **Análisis de texto**:

```python
texto = "python es genial python es poderoso"
palabras_unicas = set(texto.split())
print(f"Palabras únicas: {len(palabras_unicas)}")
```

# Resumen comparativo

| Estructura | Mutable | Ordenada | Duplicados | Indexada | Uso principal |
|---|---|---|---|---|---|
| **Lista** | Sí | Sí | Sí | Sí | Colecciones modificables |
| **Tupla** | No | Sí | Sí | Sí | Datos inmutables |
| **Set** | Sí | No | No | No | Elementos únicos |
| **Diccionario** | Sí | Sí | No (claves) | Por clave | Mapeo clave-valor |

**Guía de elección:**

- **¿Necesitas asociar clave → valor? → Diccionario**
- **¿Necesitas eliminar duplicados? → Set**
- **¿Los datos no deben cambiar? → Tupla**
- **¿Necesitas orden y modificar? → Lista**

# Ejercicios prácticos

## Ejercicio 1: Agenda de contactos

Crea una agenda que use diccionarios:

```python
contactos = {}

def agregar_contacto(nombre, telefono, email):
    contactos[nombre] = {"telefono": telefono, "email": email}

def buscar_contacto(nombre):
    return contactos.get(nombre, "No encontrado")

def mostrar_todos():
    # Implementar
    pass

# Prueba tu código
agregar_contacto("Ana", "912345678", "ana@email.com")
buscar_contacto("Ana")
```

## Ejercicio 2: Análisis de calificaciones

```python
calificaciones = {
    "Ana": [8.5, 9.0, 7.5, 8.0],
    "Carlos": [6.0, 7.0, 6.5, 7.5],
    "María": [9.5, 9.0, 9.2, 9.8]
}

# Calcula:
# 1. Promedio de cada estudiante (usa sum() y len())
# 2. Estudiante con mejor promedio (usa max())
# 3. Cuántos aprobaron (promedio >= 5)
# 4. Diccionario con solo aprobados (usa bucle for)
```

# Ejercicio 3: Contador de palabras

```python
def contar_palabras(texto):
    """
    Cuenta cuántas veces aparece cada palabra.
    Devuelve un diccionario {palabra: cantidad}
    """
    pass

texto = "python es genial python es poderoso python es facil"
resultado = contar_palabras(texto)
# Esperado: {'python': 3, 'es': 3, 'genial': 1, 'poderoso': 1, 'facil': 1}
```

# Ejercicio 4: Análisis de estudiantes con sets

```python
matematicas = {"Ana", "Carlos", "María", "Juan"}
fisica = {"María", "Juan", "Luis", "Carmen"}
quimica = {"Ana", "Juan", "Carmen"}

# Encuentra:
# 1. Estudiantes en las tres materias (usa & tres veces)
# 2. Estudiantes solo en Matemáticas (usa -)
# 3. Estudiantes en al menos dos materias
```

# Ejercicio 5: Fusionar diccionarios

```python
# Dado tres diccionarios de ventas mensuales:
ventas_enero = {"A": 100, "B": 200}
ventas_febrero = {"A": 120, "B": 180, "C": 90}
ventas_marzo = {"A": 110, "C": 160}

# Crea un diccionario 'total' que sume las ventas de cada producto
# Si un producto no está en un mes, se cuenta como 0
# Usa bucles for para recorrer cada diccionario

# Esperado: {'A': 330, 'B': 380, 'C': 250}
```

# Ejercicio 6: Invertir diccionario anidado

```python
# Dado un diccionario de estudiantes y sus ciudades
estudiantes = {
    "est001": {"nombre": "Ana", "ciudad": "Madrid"},
    "est002": {"nombre": "Carlos", "ciudad": "Barcelona"},
    "est003": {"nombre": "María", "ciudad": "Madrid"}
}

# Crea un diccionario agrupado por ciudad
# Esperado: {'Madrid': ['Ana', 'María'], 'Barcelona': ['Carlos']}
```

# Ejercicio 7: Top 3 palabras más frecuentes

```python
texto = """
Python es un lenguaje de programación interpretado cuya filosofía
hace hincapié en la legibilidad de su código. Python es muy popular.
Python es poderoso y Python es fácil de aprender.
"""

# 1. Cuenta frecuencia de cada palabra (diccionario)
# 2. Encuentra la palabra más frecuente
# 3. Muestra las palabras ordenadas alfabéticamente
```

# Ejercicio 8: Validador de contraseña

```python
def validar_contraseña(password):
    """
    Una contraseña válida debe:
    - Tener al menos 8 caracteres
    - Contener al menos una mayúscula
    - Contener al menos una minúscula
    - Contener al menos un dígito

    Usa bucles for para las validaciones.
    Devuelve (bool, str) con el resultado y mensaje
    """
    pass

# Pruebas
print(validar_contraseña("Hola123"))       # (True, "Válida")
print(validar_contraseña("hola123"))       # (False, "Falta mayúscula")
print(validar_contraseña("HOLA123"))       # (False, "Falta minúscula")
print(validar_contraseña("HolaHola"))      # (False, "Falta dígito")
```

# Ejercicio 9: Sistema de inventario

```python
inventario = {
    "PRO001": {"nombre": "Laptop", "precio": 800, "stock": 15},
    "PRO002": {"nombre": "Mouse", "precio": 20, "stock": 50},
    "PRO003": {"nombre": "Teclado", "precio": 45, "stock": 5}
}

# Implementa:
# 1. valor_total_inventario() - suma precio * stock de todos
# 2. productos_bajo_stock(minimo) - lista productos con stock < minimo
# 3. producto_mas_caro() - devuelve tupla (codigo, nombre, precio)
# 4. aplicar_descuento(porcentaje) - reduce todos los precios
```

# Ejercicio 10: Analizador de correos

```python
correos = [
    "ana@gmail.com",
    "carlos@hotmail.com",
    "maria@gmail.com",
    "juan@yahoo.com",
    "pedro@gmail.com",
    "laura@hotmail.com"
]

# Usa bucles y funciones para:
# 1. Contar cuántos correos hay por dominio (diccionario)
#    Esperado: {'gmail.com': 3, 'hotmail.com': 2, 'yahoo.com': 1}
# 2. Crear set de dominios únicos
# 3. Encontrar el dominio más popular (recorre el diccionario del punto 1)
# 4. Lista de usuarios que usan gmail (usa bucle for con filtro if)
```